Humanitarian Crisis Neglect Index With Uncertainty Quantification and Counterfactual Reasoning

Technical Documentation

Identifying Severe but Underfunded Humanitarian Crises

Team Alles GUT

Task Geo-Insight: Which Crises Are Most Overlooked?

Date April 2026

0 Contents

1 Introduction **2**
1.1 Motivation . . . 2
1.2 Design Principles . . . 2

2 Methodology **2**
2.1 Neglect Index Formula . . . 2
2.2 Severity Score . . . 2
2.3 Component Definitions . . . 3
2.3.1 need_rank . . . 3
2.3.2 ipc_severity . . . 3
2.3.3 events_rank . . . 3
2.3.4 coverage_rank . . . 3
2.4 Structural Neglect Signals . . . 4
2.5 Bootstrap Uncertainty Estimation . . . 4
2.6 Priority Labels . . . 4

3 System Architecture **4**
3.1 Backend (`api/`) . . . 4
3.2 Frontend (`frontend/src/`) . . . 5

4 API Reference **5**
4.1 `GET /metadata` . . . 5
4.2 `GET /ranking` . . . 5
4.3 `GET /tsne` . . . 6
4.4 `GET /crisis/{country}/{year}/{cluster}` . . . 6
4.5 `GET /counterfactual` . . . 6
4.6 `POST /chat` (SIDE TASK) . . . 7

5 Data Sources **7**
5.0.1 Live data (HDX HAPI) . . . 8
5.0.2 Data merging pipeline . . . 8

6 Installation & Setup **8**
6.1 Prerequisites . . . 8
6.2 Backend . . . 8
6.3 Frontend . . . 8
6.4 Data Exploration . . . 8
6.5 Environment Variables . . . 8

7 Usage Guide **9**
7.1 Ranking Crises . . . 9
7.2 Adjusting Weights . . . 9
7.3 t-SNE Visualisation . . . 9
7.4 Counterfactual Analysis . . . 9
7.5 Chatbot . . . 9

8 Limitations and Caveats **9**

9 Project Structure **10**

A Humanitarian Cluster Taxonomy **10**

1 Introduction

1.1 Motivation

Not every humanitarian crisis receives funding proportional to its documented need. Some situations are severe, chronic, and well-documented - yet year after year they receive far less than their requested requirements. Others may be similarly dire but are acute or newly escalating, making them "structurally" overlooked rather than recently neglected.

The **Humanitarian Crisis Neglect Index (HCNI)** is a data-driven platform that surfaces these overlooked crises by combining two complementary signals:

- **Crisis severity** — the scale and urgency of documented humanitarian need, derived from people-in-need counts, food-insecurity phase data, and conflict events.
- **Funding gap** — how far the current funding coverage falls short of requirements, computed from OCHA's Financial Tracking Service (FTS).

A crisis scores high on the neglect index when it is *simultaneously* severe and underfunded relative to its cluster peers. The system is designed for decision support, not automated allocation: outputs are ranked and explained so that a humanitarian analyst or donor advisor can interrogate, adjust, and act on them.

1.2 Design Principles

1. **Transparency** — every component of the score is visible and adjustable.
2. **Uncertainty awareness** — bootstrap confidence intervals surface ranking instability; missing data is flagged, not silently imputed.
3. **Within-cluster fairness** — all percentile ranks compare crises only against peers in the same humanitarian cluster (e.g. Health vs. Shelter).
4. **Structural vs. acute neglect** — a six-year FTS history distinguishes chronic underfunding from a recent deterioration.
5. **Human-in-the-loop** — the Claude-powered chatbot acts as an analytical co-pilot; it never makes allocation decisions autonomously.

2 Methodology

2.1 Neglect Index Formula

For each crisis c (identified by country, humanitarian cluster, and year), the neglect index $\mathcal{N}(c)$ is defined as:

$$\mathcal{N}(c) \;=\; \widetilde{S}_w \cdot \text{severity}(c) \;+\; \widetilde{G}_w \cdot \big(1 - \text{coverage_rank}(c)\big) \tag{1}$$

where $\widetilde{S}_w$ and $\widetilde{G}_w$ are the severity weight and funding-gap weight normalised to sum to 1:

$$\widetilde{S}_w = \frac{S_w}{S_w + G_w}, \qquad \widetilde{G}_w = \frac{G_w}{S_w + G_w}.$$

Default weights: $S_w = 0.6$, $G_w = 0.4$.

2.2 Severity Score

The severity term itself is a weighted combination of up to three signals. Exactly which signals are used depends on data availability (Cases A–D):

Case A (IPC + conflict events available)

$$\text{severity}(c) \;=\; \widetilde{N}_w \cdot \text{need_rank}(c) \;+\; \widetilde{I}_w \cdot \text{ipc_severity}(c) \;+\; \widetilde{E}_w \cdot \text{events_rank}(c)$$

Case B (IPC only)

$$\text{severity}(c) \;=\; \widetilde{N}_w \cdot \text{need_rank}(c) \;+\; \widetilde{I}_w \cdot \text{ipc_severity}(c)$$

Case C (conflict events only)

$$\text{severity}(c) \;=\; \widetilde{N}_w \cdot \text{need_rank}(c) \;+\; \widetilde{E}_w \cdot \text{events_rank}(c)$$

Case D (neither IPC nor events)

$$\text{severity}(c) \;=\; \text{need_rank}(c)$$

In all cases the sub-weights $\widetilde{N}_w$, $\widetilde{I}_w$, $\widetilde{E}_w$ are re-normalised to sum to 1 using the available components. Default sub-weights: $N_w = 0.5$, $I_w = 0.4$, $E_w = 0.1$.

2.3 Component Definitions

2.3.1 need_rank

Percentile rank of the number of *people in need* for crisis c among all crises in the same humanitarian cluster. Ranks are computed on the aggregate dataset after collapsing multi-year rows into a single (country, cluster) observation.

2.3.2 ipc_severity

The Integrated Food Security Phase Classification (IPC) records the population share in each phase 1–5 for the "Food Security" cluster. The IPC severity score maps the average phase $\bar{p}$ to a $[0, 1]$ scale with exponential weighting that penalises famine (phase 5) strongly:

$$\text{ipc_severity} \;=\; \frac{2^{\bar{p}-1} - 1}{15} \tag{2}$$

This gives ipc_severity $= 0$ at phase 1 (minimal) and ipc_severity $= 1$ at phase 5 (famine).

2.3.3 events_rank

Percentile rank of the number of ACLED conflict events targeting civilians in the country and year of crisis c, computed within the same humanitarian cluster.

2.3.4 coverage_rank

Percentile rank of the funding coverage ratio:

$$\text{coverage}(c) \;=\; \min\left(\frac{\text{funding}(c)}{\text{requirements}(c)},\; 1\right)$$

A high `coverage_rank` (coverage better than peers) produces a low contribution to the neglect index via the $(1 - \text{coverage_rank})$ term.

2.4 Structural Neglect Signals

Using six years of FTS history (2020–2025), each (country, cluster) pair is characterised by four structural signals:

`consecutive_years_underfunded` The length of the current streak of years with coverage $< 50\%$.

`structural_neglect_score`

$$\text{sn_score} = 0.6 \cdot \frac{\min(\text{streak}, 5)}{5} + 0.4 \cdot \text{pct_years_underfunded}$$

`coverage_trend` Linear regression slope of coverage over the six-year window. A negative value indicates worsening coverage.

`neglect_type` One of: **structural** ($\geq$3 consecutive years + $\geq$60% of window underfunded), **worsening** (negative trend + currently underfunded), **acute**, **improving**, or **adequate**.

2.5 Bootstrap Uncertainty Estimation

When *`n_bootstrap`* > 0 (default 200), the scoring procedure is repeated on random sub-samples of the crisis pool. The resulting rank distribution is summarised as a 90% confidence interval $[\text{rank}_{5\%}, \text{rank}_{95\%}]$. A wide interval signals that a crisis's rank is sensitive to which other crises happen to be in the dataset—a meaningful caveat for sparse clusters.

2.6 Priority Labels

Crises are labelled by neglect index thresholds (configurable):

Label	Default threshold	Interpretation
Critical	$\mathcal{N} \geq 0.80$	Immediate attention warranted
High	$0.60 \leq \mathcal{N} < 0.80$	Elevated concern
Medium	$0.40 \leq \mathcal{N} < 0.60$	Monitor closely
Low	$\mathcal{N} < 0.40$	Relatively well covered

3 System Architecture

The system follows a three-tier architecture: a data layer of static CSV/XLSX files plus a live HDX HAPI feed, a Python FastAPI backend, and a React frontend.

3.1 Backend (`api/`)

`main.py` FastAPI application. Defines all HTTP endpoints, loads the aggregate CSV into memory on startup, and wires the scorer, chatbot, and HDX client together.

`scorer.py` Core scoring engine. All neglect-index formula logic (Equation 1) lives here.

`chatbot.py` Claude API integration. Runs an agentic loop where Claude can call tools (data queries, parameter updates, HDX lookups) before returning a natural language response.

`hdx_client.py` HTTP client for the HDX HAPI. Provides eight data-fetching methods.

Listing 1: High-level architecture overview

```
+-------------------------------------------------------------+
|                 React Frontend  (:3000)                     |
|  CsvViewer . TsneChart . Chat/Weights . WorldMap            |
|                  CounterfactualSlider                       |
+----------------------+-----------------------------------+
                       |  HTTP / JSON  (:8000)
+----------------------v-----------------------------------+
|              FastAPI Backend  (:8000)                       |
|   main.py  -  chatbot.py  -  scorer.py  -  hdx_client   |
+----------------------+-----------------------------------+
        +--------------+--------------+
        |                             |
   CSV / XLSX files             HDX HAPI (live)
   (data/ directory)            api.hpc.tools
```

3.2 Frontend (`frontend/src/`)

`App.js` Root component; holds all shared state (`csvData`, `currentParams`, chat history, selected crisis).

`CsvViewer.js` Sortable, filterable data table. Supports cluster-level and country-level rollup views.

`TsneChart.js` Interactive 2D scatter plot using Recharts. Points are coloured by neglect index; KMeans cluster assignments are overlaid; outliers are marked with red rings.

`Chat.js` Chatbot panel. Renders Claude's Markdown responses and automatically applies parameter updates returned by the API.

`Weights.js` Collapsible methodology panel showing active weights as normalised bar charts.

`CounterfactualSlider.js` "What-if" overlay. A funding slider triggers a `/counterfactual` query; the resulting rank change and new t-SNE position are reflected in real time.

`WorldMap.js` Geographic view using `react-simple-maps`; hover-to-filter integration with the table.

`RadarModal.js` Detailed metrics popover for a selected crisis.

4 API Reference

All endpoints return JSON. The base URL is `http://localhost:8000`.

4.1 `GET /metadata`

Returns the set of available countries, clusters, and years in the loaded dataset.

Response:

```
{ "countries": [...], "clusters": [...], "years": [...] }
```

4.2 `GET /ranking`

The primary ranking endpoint. Accepts 25+ query parameters.

4.2.0 Filter parameters

Parameter	Type	Description
last_years	int	Only include data from the most recent N years
cluster[]	string[]	Restrict to specific humanitarian clusters
country[]	string[]	Restrict to specific country codes (ISO3)
min_people_in_need	float	Minimum people-in-need threshold
max_coverage	float	Maximum funding coverage (0–1)
min_neglect_index	float	Minimum neglect index filter
neglect_type[]	string[]	Filter by structural neglect type
min_consecutive_years	int	Minimum consecutive underfunded years

4.2.0 Weight parameters

Parameter	Default	Description
severity_weight	0.6	Weight of severity vs. funding gap
funding_gap_weight	0.4	Weight of funding gap
need_weight	0.5	Sub-weight: people in need
ipc_weight	0.4	Sub-weight: IPC food security
events_weight	0.1	Sub-weight: conflict events

4.2.0 Display and pagination parameters

Parameter	Default	Description
critical_threshold	0.8	Neglect index for "critical" label
high_threshold	0.6	Neglect index for "high" label
sort_by	`neglect_index`	Column to sort by
sort_desc	true	Descending sort
limit	50	Results per page
offset	0	Pagination offset
n_bootstrap	0	Bootstrap iterations (0 = off, max 500)

4.3 `GET /tsne`

Returns a 2D t-SNE projection of the scored crises, plus KMeans cluster assignments (2–6 clusters) and outlier flags.

4.4 `GET /crisis/{country}/{year}/{cluster}`

Returns full scoring context for a single crisis: all metrics, ranks, severity case (A/B/C/D), and bootstrap confidence bounds.

4.5 `GET /counterfactual`

Computes the hypothetical effect of additional funding on a crisis.

Query parameters: *country_code*, *cluster*, *additional_funding* (USD).

Response:

```
{
  "current_neglect_index": 0.84,
```

```
  "new_neglect_index":      0.71,
  "rank_change":            +12,
  "coverage_improvement":   0.18
}
```

4.6 POST /chat (SIDE TASK)

Runs the Claude-powered analysis pipeline.

Request body:

```
{
  "messages": [ {"role": "user", "content": "..."} ],
  "current_params": { ...active frontend weights/filters... }
}
```

Claude may call up to ten internal tools before responding:

- `query_ranking` — execute a ranking query with custom params
- `update_ranking_parameters` — push new params to the frontend
- `hdx_search_locations`, `hdx_get_humanitarian_needs`, `hdx_get_affected_populations`, `hdx_get_food_security`, `hdx_get_conflict_events`, `hdx_get_funding`, `hdx_get_operational_presen`

Response:

```
{
  "reply":              "Prose analysis...",
  "parameter_update":   { ...merged params dict or null... },
  "ranking_snapshot":   [ ...top results at time of response... ]
}
```

5 Data Sources

All primary data files reside in the `data/` directory.

Source	File(s)	Description
FTS (OCHA)	`fts_requirements_funding_global.csv`, `fts_requirements_funding_globalcluster_global`	Global funding and requirements, 2000–2025, by country, cluster, and year. Primary source for coverage calculations.
HPC / HNO	`hpc_hno_2024.csv`, `hpc_hno_2025.csv`	Humanitarian Needs Overview: people in need (PiN) by country, cluster, and year. Source for `need_rank`.
IPC (FEWS Net)	`ipc_global_area_wide.csv`	Integrated Food Security Phase Classification by country and year. Population shares in phases 1–5. Used for `ipc_severity` in the "Food Security" cluster.
ACLED	`number_of_events_targeting_civilians_....csv`	Conflict events targeting civilians, by country and year. Source for `events_rank`.

5.0.1 Live data (HDX HAPI)

When the environment variable `HDX_APP_IDENTIFIER` is set, the chatbot can query the HDX Humanitarian API (`api.hpc.tools`) for real-time data on needs, food security, conflict, funding, and operational presence.

5.0.2 Data merging pipeline

The aggregate data frame construction follows these steps:

1. Load FTS plan-level and cluster-level funding.
2. Load HNO people-in-need figures (2024, 2025).
3. Merge FTS + HNO on `(countryCode, cluster, year)`.
4. Left-merge IPC scores on `(countryCode, year)` where `cluster = "Food Security"`.
5. Left-merge ACLED civilian events on `(countryCode, year)`.
6. Left-merge structural neglect signals (computed from 6-year FTS window).

6 Installation & Setup

6.1 Prerequisites

- Python 3.11+
- Node.js 18+ and `npm`
- `uv` (Python package manager) or `pip`
- `latexmk` and a LaTeX distribution (for this document)

6.2 Backend

```
uv run uvicorn api.main:app
```

The server will be available at http://localhost:8000. Interactive API docs (Swagger) are at http://localhost:8000/docs.

6.3 Frontend

```
cd frontend
npm install
npm start   # -> http://localhost:3000
```

6.4 Data Exploration

The scripts used for data discovery and preliminary analyses was left in ‘scripts‘ directory.

6.5 Environment Variables

Variable	Required	Description
`ANTHROPIC_API_KEY`	Yes	Anthropic API key for the chatbot
`HDX_APP_IDENTIFIER`	No	Base64-encoded `app_name:app_email` for live HDX data

7 Usage Guide

7.1 Ranking Crises

After starting both services, open `http://localhost:3000`. The table on the bottom left displays all crises ranked by the neglect index using the default weights ($S_w = 0.6$, $G_w = 0.4$).

- Use the **filter bar** to restrict by text, cluster, or neglect type.
- Toggle between **cluster view** (one row per country–cluster pair) and **country view** (rolled up to country level).
- Priority labels (critical / high / medium / low) appear in the first column.

7.2 Adjusting Weights

Open the **Weights** panel to see the active weight configuration. Ask Claude in the chat panel to adjust weights (e.g., "put more emphasis on food security and reduce the conflict weight"); Claude will push updated parameters to the frontend automatically.

7.3 t-SNE Visualisation

Switch to the **t-SNE view** to see all crises projected onto a 2D plane where proximity reflects similarity of the scoring features. Points are coloured from green (low neglect) to red (high neglect). Outlier crises are ringed in red.

7.4 Counterfactual Analysis

Click any row in the table to open the **Counterfactual Slider**. Drag the funding slider to see in real time:

- The new neglect index and rank after the hypothetical funding is added.
- The new position on the t-SNE chart (where would it land in the embedding space).

The aid can be utilized for **projecting the possible improvement** caused by an increased funding.

7.5 Chatbot

The chat panel accepts natural language queries such as:

- *"Which food security crises in Sub-Saharan Africa have been underfunded for more than three consecutive years?"*
- *"Show me the top 10 most neglected health crises and explain what drives their scores."*
- *"What would happen if the Yemen Food Security cluster received an extra $200 million?"*

Claude will query the ranking engine, call HDX tools as needed, and return a prose response. If it suggests parameter changes, the frontend updates automatically.

8 Limitations and Caveats

1. **Data coverage is uneven.** IPC food-insecurity data exists only for the "Food Security" cluster. ACLED events are available at country level, not cluster level—all clusters in the same country receive the same events rank. Crises with missing requirements or needs figures are not scored.

2. **Funding data reflects reported figures, not actual spending.** FTS coverage ratios measure pledged/tracked funding against stated requirements. Under-reported funding or inflated requirements both distort the neglect index.

3. **Bootstrap intervals are approximate.** The bootstrap procedure resamples the full crisis pool, not individual data points within a crisis. Narrow clusters with few observations will exhibit wide intervals.
4. **The scoring is relative, not absolute.** A country that scores "critical" is critical *relative to its cluster peers*. A poorly covered cluster with uniformly low coverage may have many critical entries even if absolute need is modest.
5. **The tool is for decision support, not automated allocation.** Rankings should be interpreted alongside qualitative context. Do not use the neglect index as the sole basis for funding decisions.

9 Project Structure

```
datathon-un/
+-- api/
|   +-- main.py            # FastAPI application and endpoints
|   +-- scorer.py          # Neglect index scoring engine
|   +-- chatbot.py         # Claude API integration
|   +-- hdx_client.py      # HDX HAPI HTTP client
+-- frontend/
|   +-- src/
|   |   +-- App.js                    # Root component / state
|   |   +-- CsvViewer.js              # Data table
|   |   +-- TsneChart.js              # t-SNE scatter plot
|   |   +-- Chat.js                   # Chatbot panel
|   |   +-- Weights.js                # Methodology weights display
|   |   +-- CounterfactualSlider.js   # Funding scenario modal
|   |   +-- WorldMap.js               # Geographic map view
|   |   +-- RadarModal.js             # Detailed metrics popover
|   +-- package.json
+-- data/                  # CSV and XLSX data files (gitignored)
+-- scripts/
|   +-- data/
|       +-- create_big_csv.py  # Data aggregation script
+-- notebooks/             # Exploratory Jupyter notebooks
+-- docs/
|   +-- documentation.tex  # This document
|   +-- workshop_notes.md  # Datathon brief and notes
+-- pyproject.toml         # Python dependencies
+-- .env.example           # Environment variable template
```

A Humanitarian Cluster Taxonomy

The IASC cluster system organises humanitarian response into thematic areas. The following clusters appear in the HCNI dataset:

Cluster	Description
Food Security	Food assistance and livelihoods
Health	Primary and secondary healthcare
Nutrition	Acute malnutrition treatment and prevention
WASH	Water, sanitation, and hygiene
Shelter	Emergency shelter and NFIs
Protection	Safety, dignity, and rights
Education	Emergency and transitional education
Logistics	Humanitarian supply chain
Emergency Telecoms	Connectivity and ICT
Camp Coordination	Site management for displacement
Early Recovery	Restoration of basic services
Multi-Sector	Refugee response (UNHCR-led)